

CMPM 125 – Assignment 4

Team Members:

Yahir Rico (GitHub: Selcomd) - yarico@ucsc.edu

Yuval Szwarcbord (GitHub: Smoopiz) - yszwarcb@ucsc.edu

AI Implementation Approach

Our AI implementation uses the simple state machine implementation which uses enum for states and switch statements in Update which have three different states: Patrol, Investigate and Chase. Each of the guards cycle through each of these different states based on their hearing and vision detection which is handled by the logic we created inside of a switch statement in the guards Update loop.

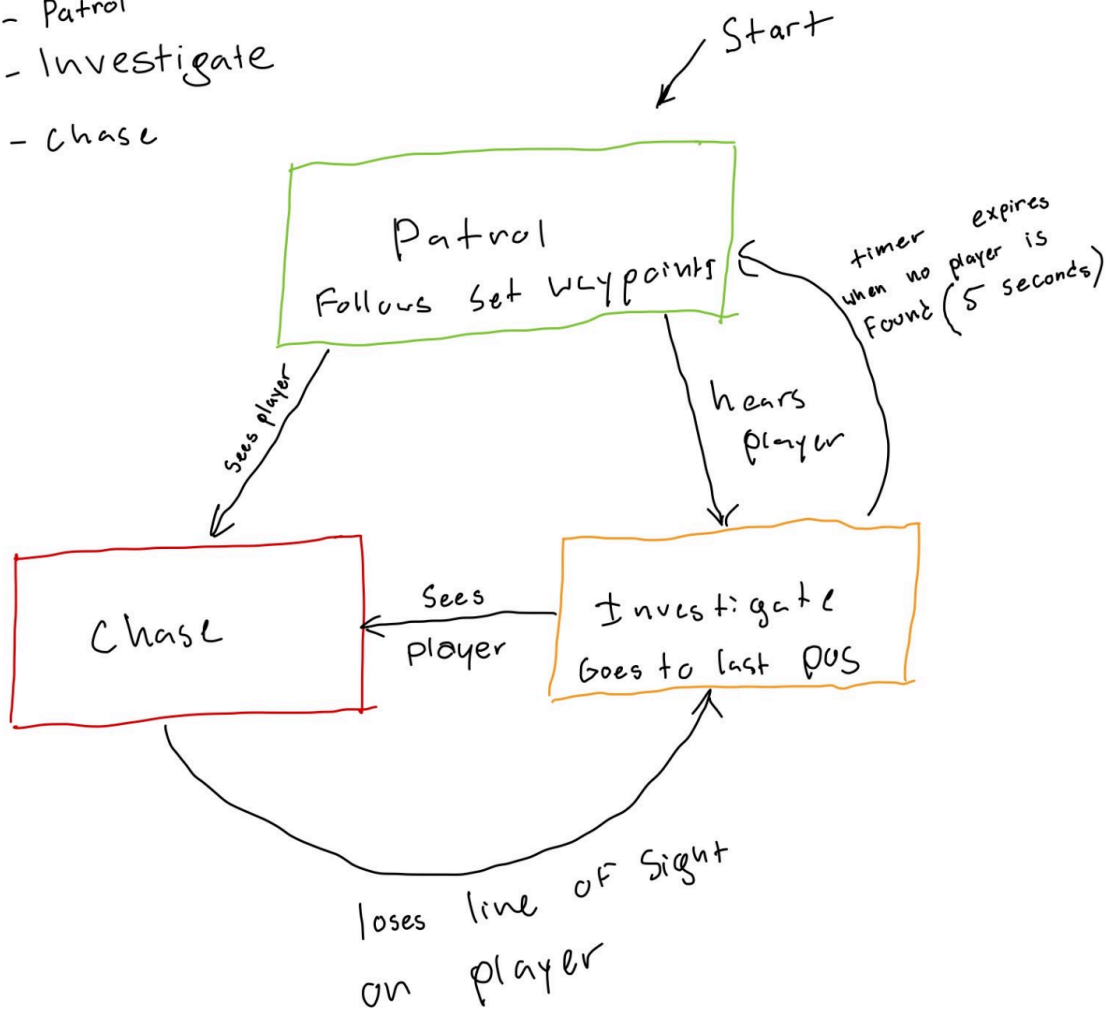
The detection is split into two different systems which is the vision which uses raycast within a defined cone and distance. This has it so if the player is within the cone and no obstacle is blocking the raycast the guard can see them. The hearing is more simple since it is just a radius check that triggers the investigation state if the player is within the circle without requiring line of sight simulating a guard reacting to a nearby sound.

Our state transitions are straightforward, we have a few guard walk around and patrol, and if they hear the player they move the player, and if they see him they start to chase them. If a chasing guard loses sight, they fall back to Investigate and head to the player's last known position. After spending a set amount of time at that last location the guard will go back to its preset patrol route. We have each guard set to change color based on its state, so the player can always know what the state of the guard is based on its look. All of our pathfinding is done with NavMesh, with each guard being given a set number of waypoint to patrol.

If a guard is able to keep the player in its vision cone for 2 seconds the loss condition is triggered, and a curtain animation plays, which in the background reloads the scene and everything that's in it.

State Machine Diagram

-  - Patrol
-  - Investigate
-  - Chase



Team Reflections

Yahir Rico:

For this assignment I contributed to building the environment by building the walls and obstacles so the players would have multiple paths they could take to get to the main objective. I also set up the obstacle colliders and multiple layers in order to properly bake the NavMesh. I also added basic guard patrol routes using waypoints and NavMeshAgents on the guards. I also implemented the first-person movement and camera controls while also adding the instructions at the beginning of the game so the player knows what the goal is. I also created the win condition where when the player gets near the main objective and presses “E” it triggers a black curtain to close off the screen and display a “You Win!” screen and then restarts the scene after a couple of seconds.

While working on this project, I learned a lot about how the NavMesh system works in unity and how important layout is for AI navigation. I also learned that obstacles need to be set up in a specific way so they can properly be baked into the NavMesh so the AI can properly navigate around them. Also, this project helped me get a better understanding of how important environment design and patrol routes are in a game that is focused around stealth.

Yuval Szwarcbord:

I was in charge of implementing the guard AI system, from building the state machine to wiring up the detection system. I started by using the patrol logic that Yahir had written down, and then on top of that I added the investigate and chase states, making sure they work well in moving from one to another before adding the vision and sound cones. They work as a functional detection system using a raycast and trigger checks, and they’re visible to the player as a mesh so they can see where the guard is looking. After all of that I created a script based off Yahir’s Win script to work for the loss, and then connect it to the guards to allow them to catch the player and reset the game.

Working on this assignment I learned in a better way what a state machine was. Before it I never even heard of what a state machine is, but after working on this assignment and connecting the state machine to the guard, it finally made me understand what a state machine is and what it does. I also learned how the choice of state machine architecture matters. We went with a simple enum and switch statement which was easy to write up and it’s more than enough for the three states that we had used, but I can see why it would be inefficient for a larger state machine. This assignment gave me a better understanding of how AI behavior in games is structured, and even how a simple state logic can create enemies that feel intelligent for players.